

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 5 Laboratuvar Föyü

Operatör Yükleme

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: C++20 derleyici (g++/clang++), bir metin editörü

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim — Neden Önemli?

Hafta 1–2'de iki noktayı toplamak için `Nokta c = topla(a, b);` yazmak zorundaydınız — matematikte en doğal yazım olan `c = a + b`'yi derleyici kendi tipiniz için tanımıyordu. Onlarca vektör işlemi olan bir kodda her toplama `topla(...)`, her çarpma `carp(...)` çağrısına dönüşür ve ifade matematiksel gösterimden uzaklaşır. Bu lab'da kendi değer tipiniz olan `Vektor2B`'yi sıfırdan oluşturacak ve onu `+`, `-`, `*`, `+=`, `==`, `<=>`, `<<`, `>>`, `[]` operatörleriyle donatarak yerleşik bir `int` kadar "doğal" davranmasını sağlayacaksınız.

1 • Ön-öğretim — Ön Bilgi

Bu föy YG I ders notu Bölüm 5 (Operatör Yükleme) üzerine kuruludur ve önceki haftaların şu kavramlarını hazır varsayar: Hafta 3 — sınıf, üye fonksiyon, örtük `this`, `private` alan ve kapsülleme; Hafta 4 — `const` üye fonksiyon doğruluğu ve değer/referans dönüş bilinci; Hafta 1–2 — `geometri` ad uzayı, `Nokta` yapısı ve serbest `topla()` fonksiyonu. Bu lab aynı `geometri` ad uzayına, `Nokta`'yı hiç değiştirmeden, kız kardeş bir tip ekler.

Hafta 1–2'de `geometri::Nokta` yapısı için iki noktayı toplayan `topla(a, b)` adında *serbest* bir fonksiyon yazmıştınız; kullanım `Nokta c = topla(a, b);` biçimindeydi. Bu okunur, ama matematiksel gösterime (`c = a + b`) yabancıdır. Bu föyde aynı `geometri` ad uzayına, `Nokta`'yı hiç değiştirmeden, kız kardeş bir değer tipi olan `Vektor2B`'yi (iki tam sayı bileşenli bir 2B vektör) sıfırdan oluşturacaksınız. Adım adım bu tipe `+`, `-`, skaler `*`, `+=`, `==`, C++20 üç yönlü karşılaştırma (`<=>`), akış (`<<`/`>>`) ve indeksleme (`[]`) operatörlerini yükleyecek; her adımda derleyip çalıştırarak `Vektor2B`'nin yerleşik bir `int` gibi "doğal" davranmaya nasıl yaklaştığını gözlemleyeceksiniz. Föyün sonunda kendi operatörünüzü ekleyeceksiniz.

Bu föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi tamamlayın (kodu yazın, ardından verilen kabuk komutunu çalıştırın), sonra kendi sonucunuzu *Beklenen* kutusuyla karşılaştırarak doğru yolda olduğunuzu hemen doğrulayın. Bir şey tutmazsa, sonraki adıma geçmeden o adımı düzeltin — her adım bir öncekinin üstüne inşa edilir. Her 3–4 adımda bir gelen *Kontrol Noktası*, o ana kadar yaptıklarınızı bir arada sınamanıza yarar. Adımlar ders notu Bölüm 5'in dört ünitesiyle sırayla ilerler; her bölümün başında ilgili üniteye bir atıf bulacak, takıldığınızda oraya dönebileceksiniz.

1 Sınıf İskeleti: Alanlar ve Kurucu

(*bkz. Ders notu Bölüm 5, Ünite 1 — Neden Operatör Yükleme? Üye vs Serbest / `friend` operatör fonksiyonu.*)

Adım 1: Proje klasörünü kurup `Vektor2B` iskeletini yazın

Ne yapacağız? `hafta05-projesi` adlı bir klasör oluşturacak, içine `vektor2b.hpp` (sınıf bildirimi: `geometri` ad uzayı, private `std::array<int, 2> bilesenler_` alanı ve `Vektor2B(int x = 0, int y = 0)` kurucu bildirimi) ile `vektor2b.cpp` (kurucu gövdesi) dosyalarını yazacaksınız.

Neden? Her değer tipi önce bir iskeletle başlar: hangi veriyi tutacağı (alanlar) ve nasıl kurulacağı (kurucu). `Vektor2B` verisini iki elemanlı bir `std::array<int, 2>` içinde tutar; bu alanı `private` yaparak kapsüllemeyi korursunuz (Hafta 3). Kurucunun iki parametresi de varsayılan değerli (`= 0`) olduğu için, ileride `Vektor2B f;` gibi argümansız bir çağrı da geçerli olur. Operatörleri henüz eklemediniz; bu adımda yalnızca üstüne inşa edeceğiniz temeli hazırlıyorsunuz.

```
42    std::array<int, 2> bilesenler_;
```

```
15    explicit Vektor2B(int x = 0, int y = 0);
```

```
10  Vektor2B::Vektor2B(int x, int y) : bilesenler_{x, y} {}
```

`#pragma once` korumasını, `#include <array>` satırını ve tüm bildirimleri `namespace geometri { ... }` bloğunun içine yazmayı unutmayın; `vektor2b.cpp` dosyasının başında `#include "vektor2b.hpp"` bulunmalıdır.

```
>_ Terminal

$ mkdir hafta05-projesi && cd hafta05-projesi
$ g++ -std=c++20 -c vektor2b.cpp -o vektor2b.o
```

Beklenen sonuç: Komut hatasız/uyarısız tamamlanır ve dizinde bir `vektor2b.o` nesne dosyası oluşur. `Vektor2B` bu noktada yalnızca verisini tutabilen ve kurulabilen bir tiptir; henüz hiçbir operatörü yoktur, dolayısıyla çalıştırılabilir bir program da yoktur.

2 Aritmetik ve Bileşik Atama Operatörleri

(*bkz. Ders notu Bölüm 5, Ünite 2 — Aritmetik ve Bileşik Atama Operatörleri.*)

Adım 2: `operator+`, `operator-` ve skaler `operator*`'ı yükleyin

Ne yapacağız? `vektor2b.hpp` dosyasına aritmetik operatörlerin bildirimlerini (`operator+`, `operator-`, üye `operator*(int)` ve `operator+=`), ayrıca sınıf dışına serbest `operator*(int, const Vektor2B&)` bildirimini ekleyeceksiniz. `vektor2b.cpp` dosyasına da `+`, `-` ve *her iki* `*` operatörünün gövdelerini yazacaksınız (`operator+=` gövdesi bir sonraki adımda gelecek).

Neden? `a + b` ifadesini derleyici `a.operator+(b)`'ye çevirir: sol işlenen (`a`) sizin tipinizse operatör bir **üye** fonksiyon olabilir, örtük `this` sol işlenendir. Bu üç operatör (`+`, `-`, üye `*`) sonucu ikisinden de bağımsız **yeni bir nesne** ürettiği için **değer** (by value) döndürür ve nesneyi değiştirmedikleri için `const` işaretlidir. `v * 3` üye `operator*` ile çalışır, ama `3 * v` ÇALIŞMAZ: `int`'in `Vektor2B` alan bir üye operatörü yoktur. Çözüm, sol işleneni `int` olan **serbest** bir `operator*(int, const Vektor2B&)`'dir; bu fonksiyon yalnızca `Vektor2B`'nin `public` üye `operator*`'ına devrettiği için `friend` olmasına GEREK YOKTUR.

```
19    Vektor2B operator+(const Vektor2B& sag) const;
20    Vektor2B operator-(const Vektor2B& sag) const;
```

Adım 2 (devamı)

```
21     Vektor2B operator*(int katsayi) const;
22     Vektor2B& operator+=(const Vektor2B& sag);
```

```
47 Vektor2B operator*(int katsayi, const Vektor2B& v);
```

```
14 Vektor2B Vektor2B::operator+(const Vektor2B& sag) const {
15     return Vektor2B(bilesenler_[0] + sag.bilesenler_[0],
16                     bilesenler_[1] + sag.bilesenler_[1]);
17 }
```

```
21 Vektor2B Vektor2B::operator-(const Vektor2B& sag) const {
22     return Vektor2B(bilesenler_[0] - sag.bilesenler_[0],
23                     bilesenler_[1] - sag.bilesenler_[1]);
24 }
```

```
28 Vektor2B Vektor2B::operator*(int katsayi) const {
29     return Vektor2B(bilesenler_[0] * katsayi, bilesenler_[1] * katsayi);
30 }
```

```
42 // friend DEGILDIR: yalniz Vektor2B'nin public operator*'ina devreder.
43 Vektor2B operator*(int katsayi, const Vektor2B& v) {
44     return v * katsayi;
45 }
```

Ortak Hata / Tuzak

`operator+`'ın dönüş tipini yanlışlıkla `Vektor2B&` (referans) yapmayın. Sonuç fonksiyon içinde oluşturulan yerel bir nesnedir; ona referans döndürürseniz, fonksiyon bitince yok olan bir nesneye **sarkan (dangling) referans** elde edersiniz — tanımsız davranış. Yeni bir değer ürettiğiniz için dönüş tipi `Vektor2B` (değer) olmalıdır.

Terminal

```
$ g++ -std=c++20 -c vektor2b.cpp -o vektor2b.o
```

Beklenen sonuç: Komut hatasız tamamlanır. Bu noktada `operator+=` yalnız bildirildi ama tanımlanmadı; `-c` yalnız derleme (bağlama yok) yaptığı için bu sorun oluşturmaz. Bir `Vektor2B`'yi henüz ekrana yazdıramadığınız için (akış operatörü Adım 5'te gelecek) sonucu gözle göremezsiniz; şimdilik temiz bir derleme yeterlidir.

Adım 3: `operator+=` ile nesneyi yerinde güncelleyin

Ne yapacağız? Bir önceki adımda *bildirdiğiniz* `operator+=` üyesinin gövdesini `vektor2b.cpp` dosyasına ekleyeceksiniz.

Neden? `operator+=`, `+`'dan farklı olarak yeni bir nesne oluşturmaz; mevcut nesnenin bileşenlerini **yerinde** günceller ve `*this`'i **referansla** (`Vektor2B&`) döndürür. Bu iki fark bilinçlidir: (1) nesneyi değiştirdiği için `const` DEĞİLDİR; (2) `*this`'e referans döndürmek `(v1 += v2) += v3` gibi zincirlemeye izin verir ve burada referans sarkmaz — çünkü döndürülen nesne yerelde oluşturulan geçici bir nesne değil, çağrının üzerinde yapıldığı var olan nesnedir.

```
34 Vektor2B& Vektor2B::operator+=(const Vektor2B& sag) {
```

Adım 3 (devamı)

```
    bilesenler_[0] += sag.bilesenler_[0];
    bilesenler_[1] += sag.bilesenler_[1];
    return *this;
}
```

Analoji

+=, biriktirme kumbarasına benzer: her katkıda yeni bir kumbara almazsınız (yeni nesne oluşturmazsınız), var olan kumbaranın üstüne doğrudan eklersiniz. `v1 + v2` ise iki ayrı kumbaranın içeriğiyle **üçüncü**, yepyeni bir kumbara doldurmaya benzer — ilk ikisi değişmeden kalır.

Terminal

```
$ g++ -std=c++20 -c vektor2b.cpp -o vektor2b.o
```

Beklenen sonuç: Komut hatasız tamamlanır. Artık `Vektor2B`'nin dört aritmetik/atama operatörünün de (+, -, *, +=) hem bildirimi hem gövdesi vardır. Çalıştırıp gözlemlemek için hâlâ bir yazdırma yolu (akış operatörü) gerekiyor; onu Adım 5'te ekleyeceksiniz.

Kontrol Noktası — devam etmeden doğrulayın

`vektor2b.hpp` dosyanızda beş operatör bildirimi (üye `operator+`, `operator-`, `operator*(int)`, `operator+=` ve sınıf dışında serbest `operator*(int, const Vektor2B&)`) ile `vektor2b.cpp`'de bunların (+= dâhil) gövdeleri bulunduğundan emin olun. Serbest `operator*`'ın sınıfın DIŞINDA, `friend` OLMADAN tanımlandığını doğrulayın — yalnız üye `operator*`'a devrediyor.

3 Karşılaştırma ve C++20 Üç Yönlü Karşılaştırma (<=>)

(bkz. Ders notu Bölüm 5, Ünite 3 — Karşılaştırma ve C++20 Üç Yönlü Karşılaştırma.)

Adım 4: `operator==` ve `operator<=>`'ı `= default` ile ekleyin

Ne yapacağız? `vektor2b.hpp` dosyasına `bool operator==(const Vektor2B&) const = default;` ve `auto operator<=>(const Vektor2B&) const = default;` bildirimlerini ekleyecek; ardından `vektor2b_demo.cpp` adlı bir `main` dosyası oluşturup iki karşılaştırmayı (== ve <) `std::boolalpha` ile yazdıracaksınız.

Neden? C++17 ve öncesinde altı karşılaştırma operatörünü (==, !=, <, <=, >, >=) tek tek yazmanız gerekirdi. C++20'nin üç yönlü karşılaştırma (spaceship) operatörü <=>, `= default` ile yazıldığında derleyiciye altı operatörü de `bilesenler_` üye sırasına göre SÖZLÜKSEL (önce 0. bileşen, eşitse 1. bileşen) türetmesini söyler. =='i ayrıca `= default`'lamak yaygın bir iyi pratiktir: yalnız eşitlik sorgusu, tam bir üç yönlü sonuç hesaplamaktan genelde daha ucuzdur. Bu iki bildirim `= default` olduğu için `.cpp`'de gövde YAZMAZSINIZ. Önemli ayrıntı: `operator==` ve `operator<` birer `bool` döndürdüğü için, henüz akış operatörünüz olmasa da **sonuçlarını** bu adımda yazdırabilirsiniz.

```
    bool operator==(const Vektor2B&) const = default;
    auto operator<=>(const Vektor2B&) const = default;
```

Test `main`'inizde şu iskeleti izleyin (kendiniz yazın): üç vektör oluşturun (`Vektor2B`

Adım 4 (devamı)

`a(2, 3);`, `Vektor2B b(2, 3);`, `Vektor2B c(4, 1);`), `std::cout << std::boolalpha;` yazın, sonra `(a == b)` ve `(a < c)` ifadelerini açıklayıcı birer etiketle yazdırın. `a < c` ifadesinin `operator<=>`'tan türetildiğine dikkat edin.

Terminal

```
$ g++ -std=c++20 vektor2b_demo.cpp vektor2b.cpp -o vektor2b_demo
$ ./vektor2b_demo
```

Beklenen sonuç: Program derlenir ve iki karşılaştırma satırı basar: `a == b` sonucu `true` (ikisi de `(2,3)`), `a < c` sonucu `true`'dur — çünkü sözlüksel sırada ilk bileşen belirleyicidir (`2 < 4`). Tek bir `<=>` tanımından `<` operatörünün bedavaya türetildiğini gözlemlemiş olursunuz.

Beklenen Çıktı

```
(2,3) == (2,3) -> true
(2,3) < (4,1) -> true
```

Dikkat

`= default` yalnızca üyelerin bildirim sırasındaki karşılaştırması anlamsal sıraya eşitse doğrudur. Örneğin bir `Kesir(pay, payda)` için üye sırasına göre karşılaştırma, kesirlerin sayısal sırasıyla örtüşmez (`1/2` ile `1/3`'ü kıyaslayın); böyle bir tipte `<=>`'ı **elle** yazmanız gerekir. `Vektor2B`'de bileşenlerin sırası zaten istenen toplam sıralamayı verdiği için `= default` yeterlidir.

Kontrol Noktası — devam etmeden doğrulayın

`./vektor2b_demo` çıktınızda her iki satırın da `true` verdiğinden emin olun. `a < c` satırı derlenmiyorsa (`no match for operator<`), büyük olasılıkla `operator<=>` bildirimini eklemeyi unuttunuz — altı karşılaştırmanın tümü ondan türetilir.

4 Akış Operatörleri ve İndeksleme: Değer Tipini Tamamlamak

(bkz. Ders notu Bölüm 5, Ünite 4 — Akış Operatörleri (`<<`/`>>`) ve İndeksleme (`[]`).)

Adım 5: `operator<<` ve `operator>>`'ı `friend` olarak yazın

Ne yapacağız? `vektor2b.hpp` içinde sınıfın içine iki `friend` bildirimi (`operator<<` ve `operator>>`) ekleyecek, `vektor2b.cpp` içine gövdelerini yazacaksınız; ardından `vektor2b_demo.cpp`'yi tam aritmetik + karşılaştırma + akış gösterimiyle genişletip çalıştıracaksınız.

Neden? `std::cout << v` ifadesinde sol işlenen `std::cout` (bir `std::ostream`) olduğu için bu operatör üye OLAMAZ (üye olsaydı sol işlenen sizin tipiniz olurdu); **serbest** bir fonksiyon zorunludur. Ayrıca `private bilesenler_` alanına doğrudan eriştiği için sınıf içinde `friend` bildirilir — bu, sınıfın bu belirli serbest fonksiyona "özel alanlarıma bakabilirsin" izni vermesidir (herkese değil, yalnız isim isim belirtilene). Her iki operatör de aldıkları akış referansını (`cikis`/`giris`) geri döndürür; bu, `std::cout << v1 << v2;` gibi zincirlemeyi mümkün kılan standart kalıptır. Demoda gerçek klavye yerine `std::istringstream` ile sabit bir metinden okursunuz, böylece çıktı deterministik olur.

Adım 5 (devamı)

```
   friend std::ostream& operator<<(std::ostream& cikis, const Vektor2B& v);
   friend std::istream& operator>>(std::istream& giris, Vektor2B& v);
```

```
std::ostream& operator<<(std::ostream& cikis, const Vektor2B& v) {
   cikis << '(' << v.bilesenler_[0] << ", " << v.bilesenler_[1] << ')';
   return cikis;
}

std::istream& operator>>(std::istream& giris, Vektor2B& v) {
   giris >> v.bilesenler_[0] >> v.bilesenler_[1];
   return giris;
}
```

`vektor2b_demo.cpp`'yi Adım 4'te başlattığınız dosyanın üzerine, şu üç bölümü içerecek biçimde tamamlayın (`main` başında `using geometri::Vektor2B;` yazın; `<sstream>` ve `<algorithm>` başlıklarını eklemeyi unutmayın):

```
   Vektor2B v1(2, 3);
   Vektor2B v2(4, 1);
   std::cout << "v1=" << v1 << ", v2=" << v2 << '\n';
   std::cout << "v1+v2=" << (v1 + v2) << '\n';
   std::cout << "v1-v2=" << (v1 - v2) << '\n';
   std::cout << "v1*3=" << (v1 * 3) << '\n';
   std::cout << "3*v1=" << (3 * v1) << '\n';
   v1 += v2;
   std::cout << "v1 += v2 sonrasi v1=" << v1 << '\n';
```

```
   Vektor2B a(2, 3);
   Vektor2B b(2, 3);
   Vektor2B c(4, 1);
   std::cout << std::boolalpha;
   std::cout << "(2,3) == (2,3) -> " << (a == b) << '\n';
   std::cout << "(2,3) < (4,1) -> " << (a < c) << '\n';

   std::vector<Vektor2B> vektorler{Vektor2B(3, 1), Vektor2B(1, 5), Vektor2B(2, 2)};
   std::sort(vektorler.begin(), vektorler.end());
   std::cout << "siralanmis: ";
   for (const auto& v : vektorler) {
      std::cout << v << ' ';
   }
   std::cout << '\n';
```

```
   Vektor2B e(1, 2);
   std::cout << "operator<< ile e=" << e << '\n';

   std::istringstream giris("7 9");
   Vektor2B f;
   giris >> f;
   std::cout << "\"7 9\" metninden okunan f=" << f << '\n';
```

```
>_ Terminal

$ g++ -std=c++20 vektor2b_demo.cpp vektor2b.cpp -o vektor2b_demo
$ ./vektor2b_demo
```

Beklenen sonuç: Artık `Vektor2B`'yi `(x, y)` biçiminde yazdırabildiğiniz için tüm aritmetik sonuçları gözle görürsünüz: `v1+v2=(6, 4)`, `v1-v2=(-2, 2)`, `v1*3=(6, 9)` ve serbest

Adım 5 (devamı)

operatörün üye ile aynı sonucu verdiğini gösteren `3*v1=(6, 9)`. `v1 += v2` sonrası `v1=(6, 4)` olur. Karşılaştırma bölümü iki `true` satırının ardından `std::sort`'un `<=>` sayesinde `{(3,1),(1,5),(2,2)}`'yi `(1, 5) (2, 2) (3, 1)` sırasına soktuğunu; akış bölümü ise `"7 9"` metninden `operator>>` ile okunan `(7, 9)`'u basar.

Beklenen Çıktı

```
v1=(2, 3), v2=(4, 1)
v1+v2=(6, 4)
v1-v2=(-2, 2)
v1*3=(6, 9)
3*v1=(6, 9)
v1 += v2 sonrasi v1=(6, 4)
(2,3) == (2,3) -> true
(2,3) < (4,1) -> true
siralanmis: (1, 5) (2, 2) (3, 1)
operator<< ile e=(1, 2)
"7 9" metninden okunan f=(7, 9)
```

Analoji

Akış operatörleri bir borudaki suyun akış yönüne benzer: `<<` değeri akışın içine "doğru akıtır" (ekrana yazdırır), `>>` değeri akıştan "dışarı çeker" (girdiden okur). Okun yönü, verinin hangi tarafa aktığını gösterir.

Adım 6: `operator[]`'i const ve non-const çiftiyle yükleyin

Ne yapacağız? `vektor2b.hpp` içine iki `operator[]` bildirimi (`int& operator[](int)` ve `int operator[](int) const`), `vektor2b.cpp` içine ikisinin gövdesini ekleyecek; ardından `vektor2b_demo.cpp`'ye bir indeksleme bölümü ekleyip programı son hâliyle çalıştıracaksınız.

Neden? `operator[]`, bir bileşene ismi yerine **konumuyla** (0 ya da 1) erişmenizi sağlar. İki aşırı yükleme yazarsınız çünkü niyetler farklıdır: non-const sürüm `int&` (referans) döndürür, böylece `d[0] = 10;` ile bileşeni **değiştirebilirsiniz**; const sürüm `int` (kopya) döndürür ve yalnız okumaya izin verir, böylece `const Vektor2B` nesnelerinde de `[]` kullanılabilir. Geçerli indeksler yalnız 0 ve 1'dir; bu ön koşul kaynak kodda yorumla belgelenir (istisna/hata yönetimi Hafta 13'ün konusudur, bu hafta ele alınmaz).

```
31    int& operator[](int indeks);
32    int operator[](int indeks) const;
```

```
49 int& Vektor2B::operator[](int indeks) {
50    // On kosul: indeks yalniz 0 ya da 1 olmalidir (hata yonetimi Hafta 13).
51    return bilesenler_[static_cast<std::size_t>(indeks)];
52 }
53
54 int Vektor2B::operator[](int indeks) const {
55    return bilesenler_[static_cast<std::size_t>(indeks)];
56 }
```

`vektor2b_demo.cpp`'ye aşağıdaki indeksleme bölümünü, akış bölümünden **önce** gelecek biçimde ekleyin:

```
42    Vektor2B d(5, 7);
```

Adım 6 (devamı)

```
43    std::cout << "d[0]=" << d[0] << ", d[1]=" << d[1] << '\n';
44    d[0] = 10;
45    std::cout << "d[0]=10 sonrasi d=" << d << '\n';
```

Analoji

İndeksleme (`[]`), bir bileşene apartman dairesi numarasıyla gitmeye benzer: 0 numaralı daireye `d[0]`, 1 numaralı daireye `d[1]` ile doğrudan ulaşırsınız — ismini değil, numarasını verirsiniz.

Terminal

```
$ g++ -std=c++20 vektor2b_demo.cpp vektor2b.cpp -o vektor2b_demo
$ ./vektor2b_demo
```

Beklenen sonuç: Programın son hâli, önceki çıktıya iki indeksleme satırı ekler: `d[0]=5, d[1]=7` (okuma, non-const nesne) ve `d[0] = 10;` atamasından sonra `d=(10, 7)` (non-const `operator[]`'in döndürdüğü referans üzerinden değiştirme). Bu iki satır, çıktıda `siralanmis` ile `operator<<` satırlarının arasında yer alır. `Vektor2B` artık `+`, `-`, `*`, `+=`, `==`, `<=>`, `<<`, `>>` ve `[]` ile donatılmış, yerleşik bir tip gibi davranan "tam" bir değer tipidir.

Beklenen Çıktı

```
v1=(2, 3), v2=(4, 1)
v1+v2=(6, 4)
v1-v2=(-2, 2)
v1*3=(6, 9)
3*v1=(6, 9)
v1 += v2 sonrasi v1=(6, 4)
(2,3) == (2,3) -> true
(2,3) < (4,1) -> true
siralanmis: (1, 5) (2, 2) (3, 1)
d[0]=5, d[1]=7
d[0]=10 sonrasi d=(10, 7)
operator<< ile e=(1, 2)
"7 9" metninden okunan f=(7, 9)
```

Adım 7: Kendiniz ekleyin: yeni bir operatör yükleyin

Ne yapacağız? Şimdiye kadar öğrendiğiniz kalıpları kullanarak, aşağıdakilerden **birini** seçip `Vektor2B`'ye kendiniz ekleyeceksiniz.

Neden? Bir operatörü sıfırdan yüklemek, üye/serbest kararı, dönüş biçimi (değer mi referans mi) ve `const` doğruluğu gibi bu haftanın kararlarını kendi başınıza vermenizi sağlar.

- Skaler bileşik atama `Vektor2B& operator*=(int katsayi)`: her bileşeni `katsayi` ile yerinde çarpıp `*this`'i referansla döndürün (`operator+=` ile aynı kalıp). `main`'de bir vektörü `v *= 2;` ile ikiye katlayıp yazdırın.
- Tekli eksi `Vektor2B operator-() const`: her iki bileşenin de işaretini tersine çeviren yeni bir vektör döndürün (değerle, const). `main`'de `-v`'yi yazdırıp `(0,0) - v` ile aynı sonucu verdiğini gözlemleyin.

Adım 7 (devamı)

İpucu

Yeni operatörün dönüş biçimini seçerken tek soru sorun: "yeni bir nesne mi üretiyorum, yoksa var olanı yerinde mi değiştiriyorum?" Yeni nesne üretiyorsa **değer** ve `const` (`operator-()` gibi); mevcut nesneyi güncelleyip zincirlenebilir olmasını istiyorsanız `*this`'e **referans** ve `const` DEĞİL (`operator*=` gibi).

Ortak Hata / Tuzak

Eklediğiniz operatörün bildirimini yalnız `.hpp`'ye koyup gövdesini `.cpp`'ye yazmayı unutmak, `-c` ile derlerken sorun çıkarmaz ama `vektor2b_demo` bağlanırken `undefined reference to 'geometri::Vektor2B::operator...'` hatası verir. Bildirim ve gövdeyi birlikte ekleyin.

Kontrol Noktası — devam etmeden doğrulayın

Son `./vektor2b_demo` çıktınızın on üç satırının bu föydeki son `ciktikutusu` ile birebir aynı olduğundan ve kendi eklediğiniz operatörün beklediğiniz sonucu ürettiğinden emin olun. `(6, 4)`, `(1, 5)` `(2, 2)` `(3, 1)` ve `(10, 7)` değerleri tutuyorsa, `Vektor2B`'yi uçtan uca doğru donatmışsınız demektir.

Sorun Giderme

Belirti	Olası neden	Çözüm
`undefined reference to ...operator+=`	`operator+=` yalnız bildirildi, gövdesi yazılmadı (ya da `vektor2b.cpp` bağlamaya dâhil edilmedi)	Gövdeyi `vektor2b.cpp`'ye ekleyin ve `g++` komutuna `vektor2b.cpp`'yi mutlaka koyun
`no match for` `operator<'` (ya da `==`)	`operator<=>` (veya `operator==`) bildirimi eksik	İki karşılaştırma bildirimini de `= default` ile ekleyin; altı operatör bunlardan türetilir
`3 * v1` derlenmiyor, `v1 * 3` çalışıyor	Yalnız üye `operator*(int)` var; serbest `operator*(int, const Vektor2B&)` eksik	Sınıf dışına serbest `operator*`'ı ekleyin (`friend` gerekmez, üye `operator*`'a devreder)
`operator<<` "must take exactly one argument" / üye olarak reddediliyor	`operator<<` sınıfın üyesi olarak yazıldı	Onu sınıf dışında serbest fonksiyon yapın; `private` erişim için sınıf içinde `friend` bildirin
`d[0] = 10;` derlenmiyor (`lvalue required`)	non-const `operator[]` `int` döndürüyor (kopya), referans değil	non-const sürümün dönüş tipini `int&` yapın; const sürüm `int` döndürmeye devam eder
Çıktıda vektörler `(x, y)` yerine bir adres/çöp basıyor	`operator<<` gövdesi bileşenleri değil nesnenin kendisini yazdırıyor olabilir	Gövdede `v.bilesenler_[0]` ve `v.bilesenler_[1]`'i `'('`, `", "`, `')'` ile çevreleyip yazdırdığınızı doğrulayın

5 • İzleme Aktarım — Başka Nerede?

`Vektor2B`'yi uçtan uca donatırken kullandığınız karar kalıpları tek bir tipe özgü değildir; herhangi bir değer tipine taşınır:

- **Kitaplık projesine:** Hafta 3'ün `kitaplik::Kitap`'ı şu an yalnız bir `yazdir()` üye fonksiyonuna sahip. Bu föyde öğrendiğiniz serbest/`friend` deseniyle `Kitap`'a `yazdir()`'in yerini alacak bir `operator<<`, başlığa/yıla göre sıralama için de bir `operator<=>` eklenebilir. Dikkat: `Kitap`'ın üye sırası (ör. önce `isbn`) istediğiniz sırayı vermeyebilir; o zaman `<=>`'ı elle yazmanız gerekir (bu haftanın Ünite 3 tuzağı).
- **Başka değer tiplerine:** Bir `Kesir(pay, payda)` için `operator+` yeni bir *sadeleştirilmiş* kesir oluşturup değerle döndürür, `operator+=` var olanı yerinde günceller; ama `<=>`'ı `= default` yaparsanız `1/2 < 1/3` yanlış çıkar — çapraz çarpma ile elle yazmanız gerekir. Bir `Para` ya da `Tarih` tipinde de "sol işlenen sizin tipiniz mi?" sorusu operatörün üye mi serbest mi olacağını belirler.
- **İleriye:** "Değer tipini operatörlerle tamamlama" beceriniz Hafta 6'da `Materyal` hiyerarşisi kurulurken doğrudan yeniden kullanılır.